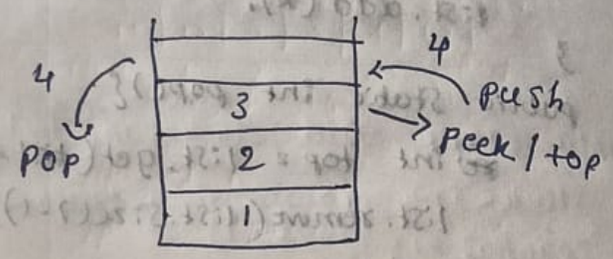
## * Stack :

Stack is a linear datastructure that stores elements in a specific order, called LIFO. means last element added first one to be removed.

real time applications : undo, redo in text editor, chrome history function call stack

operations : push O(1) peek O(1).
pop O(1)




implementation :

- Array
- LinkedList
- ArrayList

- Arrays : size is fixed so it is quite -ve for stack implementation because every time while pushing elements (new) must check stack is full / not.

```
private int[] arr;
private int top, maxSize;
public stack(int size){
maxSize = size
arr = new int[size];
top = -1;
}
```

push :
```
public void push (int x){
    if (top == maxSize-1){
        S.o.p("Stack overflow");
    }else{
        arr[++top] = x;
    }
}
```

pop :
```
public int pop(){
    if(top == -1){
        S.o.p("Stack underflow");
    }else{
        return arr[top--];
    }
}
```

peek :
```
public int Peek(){
    if(top == -1){
        S.o.p(" underflow");
    }else{
        return arr[top];
    }
}
```

- ArrayList : with dynamic size, very mush use full.

```
P.S.V.m( ){
    Stack s = new Stack();
    s.push(20);
    s.push(30);
    s.push(40);
    while(!s.isEmpty()){
        System.out.println(s.peek());
        s.pop();
    }
```

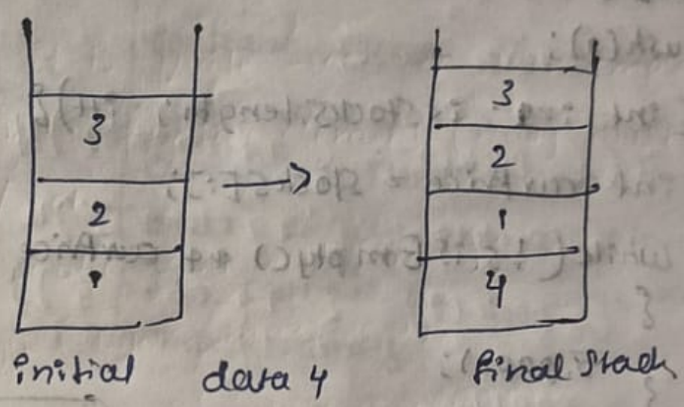
• Stack implementation using collection framework:

→ JCF :- Java collection framework it is a set of interfaces, classes, algorithms provided in Java util package.

```
// single line
Stack<Integer> s = new Stack<>(); // add this single line
s.push(10);
s.push(20);
```
} remaining same

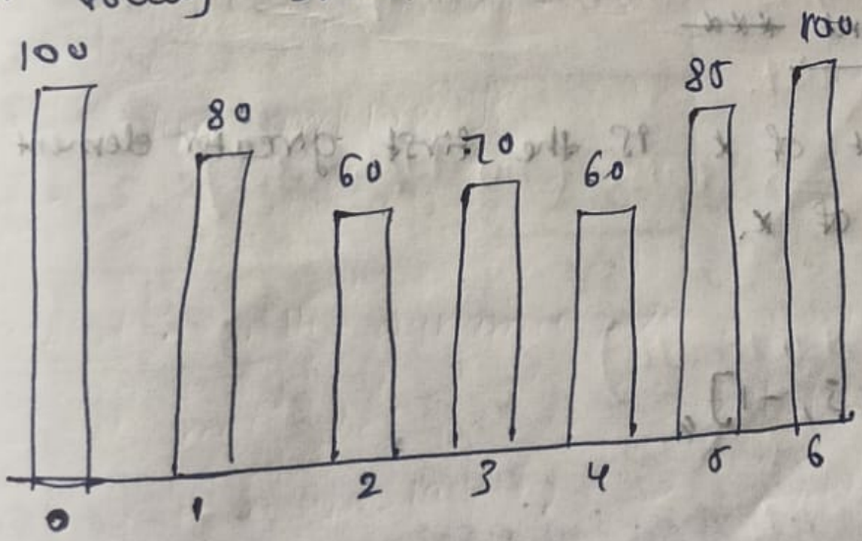
* Q1 push at the bottom of the stack.

```
public static void pushAtbottom(Stack<Integer> s, int data)
{
    if(s.isEmpty())
    {
        s.push(data);
        return;
    }
    int top = s.pop();
    pushAtbottom(s, data);
    s.push(top);
}
```




* Q2 Stock span problem:

calculating the difference / vary of stock span before & today how much it varyed. when only ~~cur~~ it should greater than today span.




| Span | 1 | 1 | 1 | 2 | 1 | 5 | 6 |
|---|---|---|---|---|---|---|---|

spaces

```
main(){
    int[] Stocks = { 100, 80, 60, 70, 60, 85, 100};
    int[] span = new int[ stocks_length];
    StockSpan(stocks, span);
    for( int i=0; i< span.length ; i++)
    {
        Sys.O.P(span[i] + " ");
    }
}
public static void StockSpan(Stocks, span){
    Stack<Integer> s = new Stack<>();
    span[0] = 1;
    s.push(0);
    for( int i=1; i<stocks.length; i++){
        int curPrice = stocks[i];
        while( !s.isEmpty() && curPrice > stocks[s.peek()])
        {
            s.pop();
        }
        if( s.isEmpty()){
            span[i] = i+1;
        }
        span[i] = i - s.peek();
        s.push(i);
    }
}
```

$\boxed{O(n)}$

Q4 Valid parenthesis;

```
public static boolean isValid(String str){
  Stack<Character> s = new Stack<>();
  for(int i=0; i< str.length; i++) {
    char ch = str.charAt(i);
    if(ch == '(' || ch == '[' || ch == '{')
    {
      s.push(ch);
    }
    else {
      if(s.isEmpty())
      {
        return false;
      }
      if((s.peek() == '(' && ch == ')') ||
        (s.peek() == '[' && ch == ']') ||
        (s.peek() == '{' && ch == '}'))
      {
        s.pop();
      }
      else{
        return false;
      }
    }
  }
  if(s.isEmpty())
  {
    return true;
  }
  else{
    return false.
  }
}
```

$O(n)$

$O(1)$

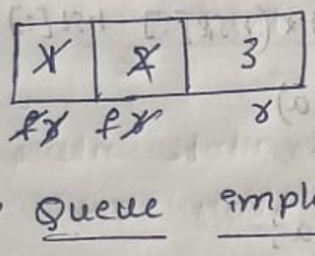
## * Queue :

queue is a linear data structure that follows (FIFO) first in first out principle, where elements are inserted at the rear & removed from the other front.

-> operations :

add / enqueue O(1)
remove / dequeue O(1)
} because always insertion from rear & deletion from front, no traversing in that case.

peek / front O(1)




-> Queue implementation :

- array
- linkedlist
- Stack.

1) array :

drawback with array
- fixed size "n" (no possible to add more elements than the size)
- remove O(n)

```
// user defined class
public static class Queue {
    Static int[] arr;
    Static int size;
    Static int rear;
    // constructor
    Queue (int n){
        arr = new int[n];
        size = n;
        rear = -1;
    }
```

```
// add
public static void add (int data) {
    if (rear == size-1)
    {
        S.o.p(" overflow");
    }
    rear = rear+1;
    arr[rear] = data;
}
// isEmpty
public static boolean isEmpty() {
    return rear == -1;
}
// peek.
public static int peek() {
    if (isEmpty) {
        S.o.p("—");
    }
    return arr[0];
}
```

```
// remove
public static int remove(){
    if(isEmpty()){
        S.O.P("not possible");
    }
    int front = arr[0];
    for(int i=0; i<rear; i++)
    {
        arr[i] = arr[i+1];
    }
    rear = rear-1;
    return front;
}
```

add → O(1)

remove → O(n) // for loop

peek → O(1)

→ circular queue using array (implementation).

to reduce remove time complexity from O(n) → O(1)

formula → rear = (rear+1) % size;

front = (front+1) % size;

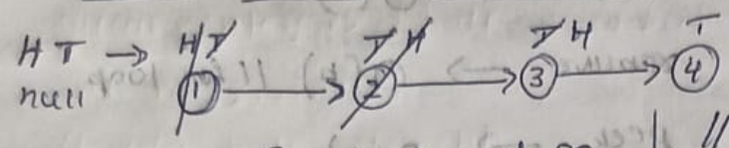
changes in the existing code:

```
public static class Queue{
    static int front;
    Queue(____){
        front = -1;
    }
}

// isEmpty
return rear==-1 && front==-1;

// isFull
public static boolean isFull(){
    return (rear+1)%size == front;
}

// add
if(isFull()){
    ____
}
if(front==-1){
    front=0;
}
```

```
// remove
int result = arr[front];
if(front == rear) // last element to delete
{
    front = -1;
    rear = -1;
}
else{
    front = (front+1)%size;
}
return result;

//peek
return arr[front];
```

all the operations O(1).

→ queue using Linkedlist:




```
class Demo { //main class

    static class Node {
        Node next;
        int data;
        Node(int d) {
            data = d;
            next = null;
        }
    }

    public static class Queue {
        static Node head = null;
        static Node tail = null;

        public static boolean isEmpty() {
            return head == null && tail == null;
        }

        // insertion
        public static void add(int data) {
            Node newNode = new Node(data);
            if(isEmpty()) {
                head = tail = newNode;
                return;
            }
            tail.next = newNode;
            tail = newNode;
        }

        // remove
        public static int remove() {
            if(isEmpty()) {
                s.o.p("nothing");
                return -1;
            }
            int result = head.data;
            if(head == tail) {
                head = tail = null;
            }
            else {
                head = head.next;
            }
            return result;
        }

        // peek
        public static int peek() {
            if(isEmpty()) {
                return -1;
            }
            return head.data;
        }
    }

    main() {
        Queue q = new Queue();
        q.add(3); // 3, 4, 5
        q.add(4);
        q.add(5);
```

peek = 3 (Head)

all the operations execute in O(1) constant time complexity.
because we are not ~~printing~~ traversing just one time incrementing head & tail doing operation.
head & tail both storing references of node object.

→ Queue using collection framework:

```
import java.util.*;
Queue<Integer> q = new LinkedList<>(); // ArrayDeque. (can use)
```

why linedlist ?

in java Queue is an interface, cannot possible to create object of interfaces. why you will learn in oops. that's why we are using classes which implements Queue. O(1)//.

→ printing first non-repeating char in a stream using Queue.

```
public static void firstNonRepeating(String str){
    Queue<Character> q = new LinkedList<>(); // import java.util.*;
    int[] freq = new int[26];
    for(int i=0; i<str.length; i++){
        char ch = str.charAt(i);
        q.add(ch); <--> freq[ch - 97]++;
        while(!q.isEmpty() && freq[q.peek - 'a'] > 1){
            q.remove();
        }
        if(q.isEmpty())
        {
            S.O.P.(-1+" ");
        }
        else{
            S.O.P(q.peek()+" ");
        }
    }
}
```

str = "aabccxb"

o/p = a -1 b b b b x

O(n)//.

while loop runs only one time.

→ InterLeaves queue (even length)

ex: q = | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |  o/p = | 1 | 6 | 2 | 7 | 3 | 8 | 4 | 9 | 5 | 10 |

q = | 1 | 2 | 3 | 4 |  o/p = | 1 | 3 | 2 | 4 |

q = | 1 | 2 | 3 | 4 |  q = | 3 | 4 | 1 | 3 | 2 | 4 |  print

half | 1 | 2 |  // one more queue additional

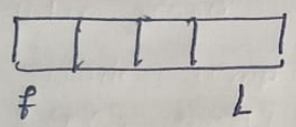
divide & store half elements.

time = O(n)

space = O(n).

```
public static void interLeaves(Queue<Integer> q){
    Queue<Integers> Fhalf = new LinkedList<>();
    int n = q.size();
    // add half elements.
    for(int i=0; i<n/2; i++){
      {
        Fhalf.add(q.remove());
      }
    // until Fhalf becomes empty.
    while(!Fhalf.isEmpty()){
        q.add(Fhalf.remove());
        q.add(q.remove());
    }
    main()
      while(!q.isEmpty()){
    }
```

* Double-ended queue (deque)




addFirst() removeFirst() getFirst()

addLast() removeLast() getLast()

- more flexible & efficient all operation $\boxed{O(1)}$

implementation of deque using Java collection framework.

```
import Java.util.*;
Deque<Integer> deque = new LinkedList<>();
```

interface

```
dq.addFirst(1); // 1
dq.addFirst(2); // [2 1]
dq.addLast(3); // 2 1 3
dq.addLast(4); // 2 1 3 4
dq.add(5);     // 2 1 3 4 5
```

dq.add(); this in which line you write there the element insert.

```
dq.removeFirst();
dq.removeLast()
// ~~2~~ 1 3 4 ~~5~~
```